



ATHLOS FORGE

BY SEBAS

PROYECTO FINAL 2º DAW
SEBASTIAN BECERRA OSPINA

Contenido

RESUMEN	4
1. INTRODUCCIÓN Y CONTEXTO DEL PROYECTO	5
1.1. Resumen del Proyecto.....	5
1.2. Objetivos del Proyecto.....	5
1.3. Análisis de Público y Mercado	6
2. ANÁLISIS DE MERCADO Y COMPETENCIA.....	6
2.1. Fuertafit.com (Sergio Peinado).....	6
2.2. ChuyAlmada.com	7
2.3. Conclusión del Análisis	8
3. Identidad de marca y diseño (branding)	9
3.1. Concepto Visual: Estética	9
3.2. Proceso de creación del logotipo	10
3.3 Sostenibilidad y valores.....	12
4. Planificación y Estructura	12
4.1. Evolución del prototipado.....	12
4.2. Wireframes y Justificación: Diseño consolidado en Figma	15
4.3. Justificación del Boceto Elegido.....	15
4.4. Requisitos y Modelado del Sistema	16
4.4.1. Requisitos Funcionales (RF)	16
4.4.2. Requisitos No Funcionales (RNF).....	16
4.5. Landing page.....	17
4.6 FACTIBILIDAD Y RECURSOS	19
4.1. Infraestructura Técnica (Software)	19
4.2. Inversión Operativa (Hardware y Logística).....	19
4.3 Cronograma y desarrollo	20
5. Implementación Técnica (Desarrollo).....	21
5.1. Stack Tecnológico y Arquitectura	21
5.2. Implementación del carrito de la compra.....	21

5.2.1. Descripción General.....	21
5.2.2. Arquitectura de Datos.....	22
5.2.3. Estándares de Manipulación del DOM.....	22
5.2.4. Funcionalidades Principales.....	23
5.2.5 Persistencia y Estado.....	23
5.3 Sistema de filtrado local.....	23
5.3.1. Estrategia de Implementación: Client-Side vs AJAX.....	23
5.3.2. Estructura del Catálogo (Grid System).....	24
5.3.3. Lógica del Buscador (Vanilla JS).....	24
5.3.4. Integración con el Carrito de Compra.....	24
5.3.5. Sistema de Comercio Electrónico y Carrito en Tiempo Real.....	25
5.3.6. Panel de Administración (Admin Dashboard).....	26
5.3.7. Gestión de Usuarios y Seguridad.....	27
5.3.8 Análisis de Ventas y Productos Vendidos.....	27
6. Sistemas de validaciones y seguridad.....	28
6.1 Validación de Identidad y Formato.....	28
6.2 Lógica de Seguridad de Contraseñas.....	28
6.3 Validación de Datos Sensibles (Tarjeta de Crédito).....	29
6.4 Gestión de Errores y Feedback.....	29
7. TESTS FUNCIONALES AUTOMATIZADOS (SELENIUM).....	29
7.1. Descripción del Entorno de Pruebas.....	30
7.2. Flujo de Ejecución Automatizado.....	30
7.3. Escenario de Prueba: Registro Múltiple.....	30
7.4. Guía de Ejecución Técnica y Despliegue de Tests.....	31
7.4.1. Requisitos Previos del Sistema.....	31
7.4.2. Automatización del Entorno (PowerShell).....	31
7.4.3. Interpretación de Resultados.....	32
8. AUDITORÍA Y VERIFICACIÓN DE ACCESIBILIDAD (WCAG 2.1).....	32
8.1. Metodología de Evaluación.....	32
8.2. Registro de Errores Detectados y Subsanaos.....	32
8.3. Mejoras en la Interfaz Dinámica (ARIA).....	33
8.4. Matriz de Conformidad Final.....	33

9. INFORME DE VERIFICACIÓN DE LA USABILIDAD: ATHLOS FORGE	34
9.1. Método: Inspección Heurística	35
9.2. Método: Test A/B	36
10. Guía de Despliegue, manual de instalación y entorno de pruebas.....	36
10.1. Estructura de Directorios del Proyecto	36
10.2. Requisitos Previos del Sistema	38
10.3. Procedimiento de Despliegue de la Aplicación.....	38
10.3.1. Versión Monolítica Tradicional (HTML5 / JavaScript ES6+ / PHP)	38
10.3.2. Versión de Arquitectura Desacoplada (Vite + React 19).....	39
10.4. Protocolo de Pruebas Automatizadas y QA (Selenium + Pytest)	40
11. Proyecto alternativo Athlos App.....	40
12. CONCLUSIÓN FINAL	43
12.1. El Desafío de la Autogestión Unipersonal.....	43
12.2. Resiliencia ante el Error	43
12.3. Potencial Laboral y Académico	44
12.4. Líneas de Trabajo Futuras y Propuestas de Mejora.....	44
12.4. Reflexión Final.....	44
13. BIBLIOGRAFÍA Y WEBGRAFÍA	45
13.1. Frameworks y Lenguajes de Programación	45
13.2. Diseño y Prototipado (UI/UX)	45
13.3. Calidad, Accesibilidad y Testing	45
13.4. Recursos Multimedia y Activos.....	46
14. Glosario Tecnico	46

RESUMEN

El presente Proyecto Fin de Grado comprende el diseño, desarrollo e implementación de **Athlos**, un ecosistema digital avanzado orientado al sector del entrenamiento personal y el fitness premium. El proyecto se aborda desde una perspectiva full-stack dividida en dos grandes bloques tecnológicos. En primer lugar, se ha desarrollado una *landing page* de alto impacto destinada a la captación de *leads* y conversión optimizada mediante arquitectura de cliente nativa (Vanilla JS ES6+), manipulación segura del DOM y diseño adaptivo robusto con Bootstrap 5. En segundo lugar, se detalla la arquitectura de **Athlos App**, una aplicación web híbrida y móvil multiplataforma desarrollada con React 19, Vite , integrada con una base de datos en MySQL. El proyecto ha sido sometido a un riguroso marco de calidad que incluye pruebas de regresión automatizadas mediante Selenium WebDriver, inspección heurística de usabilidad y una auditoría de accesibilidad conforme a los estándares WCAG 2.1 Nivel A utilizando etiquetas y estados dinámicos ARIA.

LISTADO DE FIGURAS

- **Fig 1.** Primer borrador descartado por falta de estructura clara.
- **Fig 2a.** Boceto Final Consolidado (Vista Escritorio y Móvil).
- **Fig 2b.** Boceto Final Consolidado Figma (Vista Escritorio y Móvil).
- **Fig 3.** Implementación final de la Landing Page en navegador.
- **Fig 4.** Implementación final de la Landing Page en móvil.
- **Fig 5.** Implementación final del formulario de registro y login.

1. INTRODUCCIÓN Y CONTEXTO DEL PROYECTO

1.1. Resumen del Proyecto

El proyecto Athlos Forge by Sebas consiste en el diseño e implementación de una *landing page* de alto impacto para la captación de *leads* (clientes potenciales). Este sitio web sirve como fase de prelanzamiento para la plataforma completa prevista para 2026. El objetivo principal es proyectar una imagen de profesionalidad, garantizando una experiencia de usuario (UX) fluida y una llamada a la acción (CTA) directa y eficaz.

1.2. Objetivos del Proyecto

Para guiar el desarrollo técnico y metodológico de la plataforma Athlos, se definen los siguientes objetivos:

- **Objetivo General:**

Diseñar, codificar y auditar un ecosistema digital full-stack (web corporativa) de alto rendimiento para la automatización de la captación de clientes y la autogestión de planes de entrenamiento personalizado, garantizando la resiliencia del software bajo pruebas automatizadas y estándares internacionales de accesibilidad.

- **Objetivos Específicos:**

1. Desarrollar una interfaz de usuario (UI) responsiva bajo la metodología *Mobile-First* utilizando Bootstrap 5, reduciendo la carga cognitiva y optimizando la velocidad de carga.
2. Implementar un módulo de comercio electrónico y cesta de la compra reactivo empleando exclusivamente JavaScript Vanilla (ES6+), aplicando políticas estrictas de sanitización del DOM para mitigar vulnerabilidades de inyección de código.
3. Construir una suite de pruebas funcionales automatizadas con Selenium WebDriver en Python para validar flujos críticos de autenticación y registro de datos.

4. Evaluar y corregir la interfaz interactiva mediante herramientas de auditoría (*WAVE*) para alcanzar la matriz de conformidad con las pautas WCAG 2.1 Nivel A.

1.3. Análisis de Público y Mercado

Tras un análisis detallado del sector y sus carencias, se ha determinado que Athlos Forge no se limita a un rango de edad específico. El nicho de mercado abarca a cualquier individuo que busque mejorar su calidad de vida, potenciar su rendimiento físico o recuperarse de lesiones.

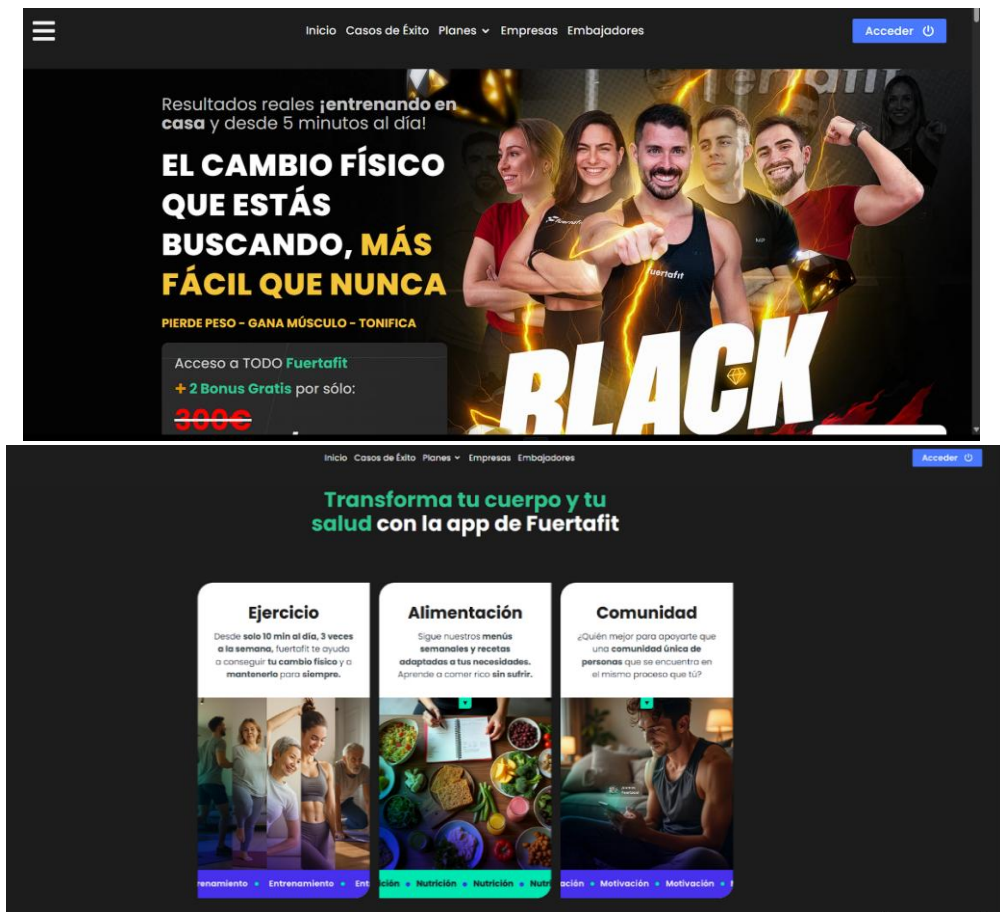
- Público Objetivo: Personas con interés en la salud integral, desde jóvenes en fase de fortalecimiento hasta adultos que buscan "honrar su vejez" a través del ejercicio.
- Líneas de Negocio:
 - B2C (Cliente final): Particulares que buscan entrenamiento personalizado o grupal de alta calidad.
 - B2B (Formación): Posibilidad de establecer cursos de capacitación para monitores del sector fitness.

2. ANÁLISIS DE MERCADO Y COMPETENCIA

Para definir la estrategia visual y de contenido, se realizó un análisis exhaustivo de dos competidores directos en el sector del fitness online: Fuertafit y Chuy Almada.

2.1. Fuertafit.com (Sergio Peinado)

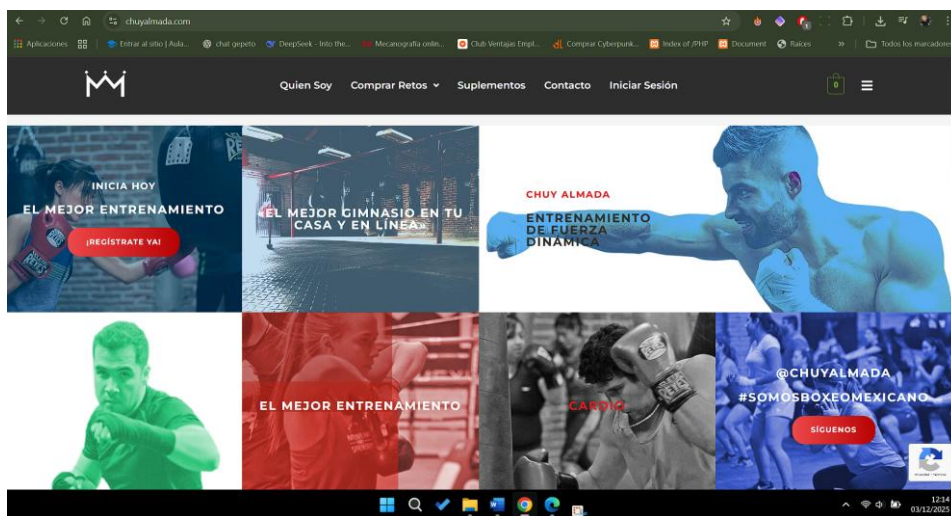
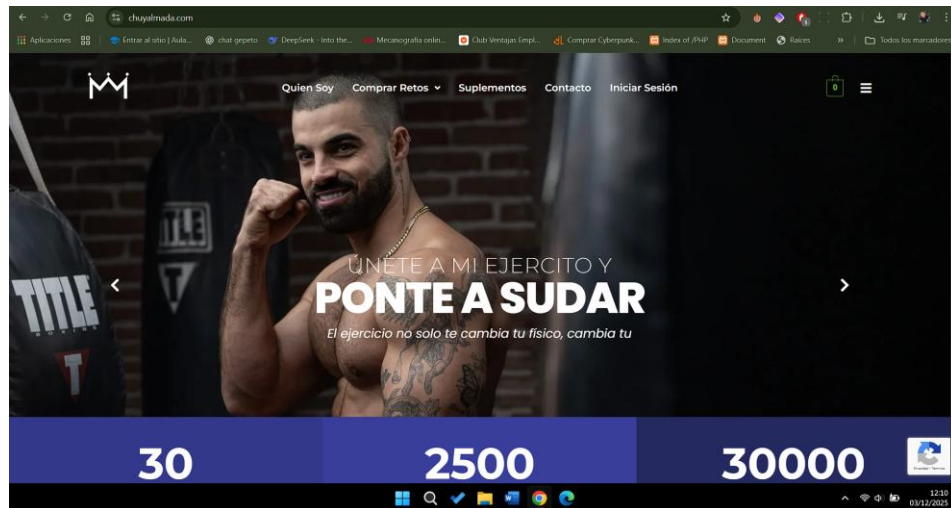
- Propuesta de Valor: Se centra en la transformación ("cambia tu cuerpo y tu vida"), ofreciendo programas premium, comunidad y resultados visibles.



- Análisis Visual (Gestalt):
 - Figura-Fondo: Uso excelente del contraste con bloques claros sobre fondos limpios.
 - Proximidad: Agrupación lógica de textos y botones para facilitar la lectura.
 - Jerarquía: Títulos grandes y subtítulos que guían la mirada sin saturar.
- Uso del Color: Predominio de azules (confianza) y blancos (limpieza), utilizando el amarillo cálido exclusivamente para los botones de acción (CTA), destacándolos sin ser agresivos.

2.2. ChuyAlmada.com

- Propuesta de Valor: Marca personal muy fuerte basada en intensidad, motivación y disciplina ("entrena o revienta").



- **Análisis Visual:** Diseño más agresivo y dinámico. Utiliza contrastes fuertes, fondos oscuros y tipografías gruesas para generar impacto inmediato.
- **Uso del Color:** Fondos oscuros (fuerza) combinados con rojos o naranjas para los CTA, colores asociados a la acción y la urgencia.

2.3. Conclusión del Análisis

Se detectó que, aunque ambos competidores son efectivos, sus menús de navegación ocupan demasiado espacio y, en ocasiones, la saturación visual puede distraer. Decisión de Diseño: Athlos Forge combinará la limpieza estructural de Fuertafit con la fuerza visual y los fondos oscuros de Chuy Almada, buscando una identidad propia basada en la elegancia ("Premium Spartan").

3. Identidad de marca y diseño (branding)

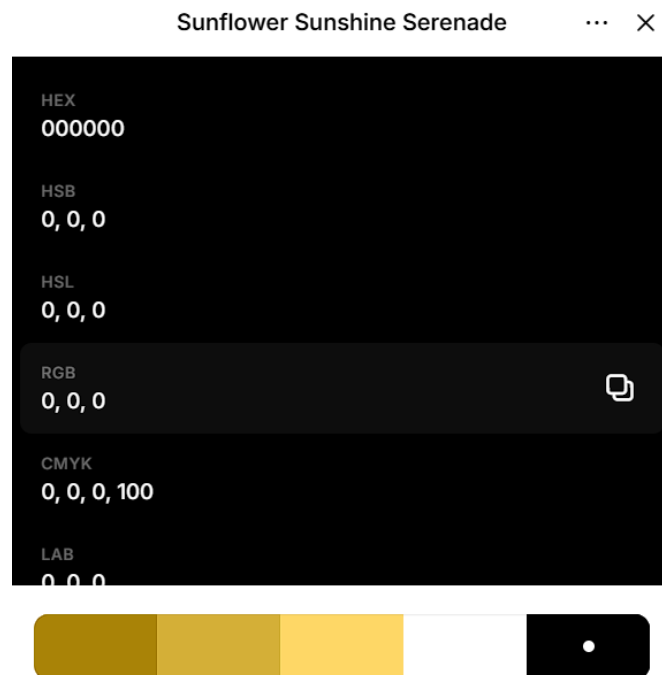
3.1. Concepto Visual: Estética

Al ubicarse el área de influencia en Las Rozas, una zona de renta per cápita elevada, la identidad visual debe alinearse con un estándar de exclusividad.

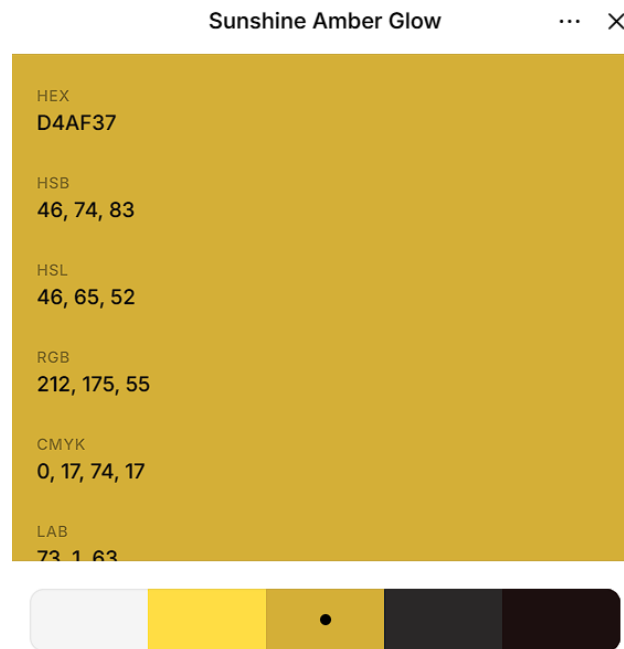
- **Paleta de Colores:** Se ha seleccionado el dorado sobre fondo negro. El negro aporta elegancia y sobriedad, mientras que el dorado resalta la calidad "premium" y capta la atención de forma aspiracional.
- **Tipografía y Estilo:** Se utilizarán fuentes audaces y limpias que refuercen la idea de resistencia y longevidad.

La elección de los colores además de ser estética, ha sido estratégica, basándome en la psicología del color para reforzar el mensaje de la marca "Athlos Forge":

- **Negro Profundo (#000000):** Se seleccionó como base estructural para transmitir autoridad, elegancia y exclusividad. A diferencia del blanco, el fondo oscuro reduce la fatiga visual y permite que los elementos de acción resalten con mayor intensidad, evocando la seriedad de una "forja" de entrenamiento.



- Dorado Espartano (#D4AF37): Se substituyó el amarillo estándar de la competencia por un tono dorado metálico. Este color simboliza éxito, calidad y energía. Funciona como guía visual para el usuario, dirigiendo la mirada instintivamente hacia los puntos clave de conversión (títulos y botones CTA).



La combinación de ambos genera un alto contraste que mejora la legibilidad y transmite una imagen "Premium Spartan", diferenciándose de la estética agresiva o caótica de otros competidores.

3.2. Proceso de creación del logotipo

El desarrollo del logotipo atravesó varias fases iterativas para encontrar el equilibrio entre la intensidad del entrenamiento y la limpieza digital:

1. Exploración: Se probaron diseños tipo "escudo" (badge) y siluetas realistas, pero se descartaron por perder legibilidad al reducirse en la barra de navegación móvil, la aplicación utilizada para el diseño fue Looka, ya que de forma gratuita genera logos a partir de diferentes logos ya establecidos.



2. Selección Final (Minimalismo): Se optó por un isotipo lineal de un corredor en posición de impulso (sprint).



Justificación: El diseño minimalista elimina el ruido visual, permitiendo que el logo sea reconocible instantáneamente tanto en una pantalla de escritorio grande como en un dispositivo móvil pequeño.

Tipografía: Acompañado de una fuente Sans-Serif geométrica en mayúsculas, el conjunto es fácil de leer y se integra orgánicamente en la barra de navegación sin competir con el resto del contenido.

Para la eliminación del fondo, he utilizado una página muy curiosa que se llama dverso laundry, que de una forma muy “kawai” , te elimina el fondo de logo.



3.3 Sostenibilidad y valores

Siguiendo las pautas del proyecto, Athlos Forge integra la sostenibilidad no solo como concepto ecológico, sino aplicado al bienestar humano:

- Entrenamiento Sostenible: Fomento de hábitos a largo plazo frente a resultados fugaces ("Reto de movilidad sostenible", "Entrena para la longevidad").
- Material reciclado: Se promueve la utilización de elementos cotidianos como botellas de agua, garrafas o mochilas llenas de libros, para simular el peso que podría llevar en el gimnasio. Ahorrando en material, y evitando el uso desmedido de material deportivo.
- Recursos: Promoción de entrenamientos que no requieren equipamiento excesivo, aprovechando el peso corporal y el entorno.

4. Planificación y Estructura

4.1. Evolución del prototipado

Se partió de un primer boceto individual que, tras ser revisado, presentaba carencias en armonía visual y jerarquía además de no contar de un CTA claro que favorezca la captación de los clientes.

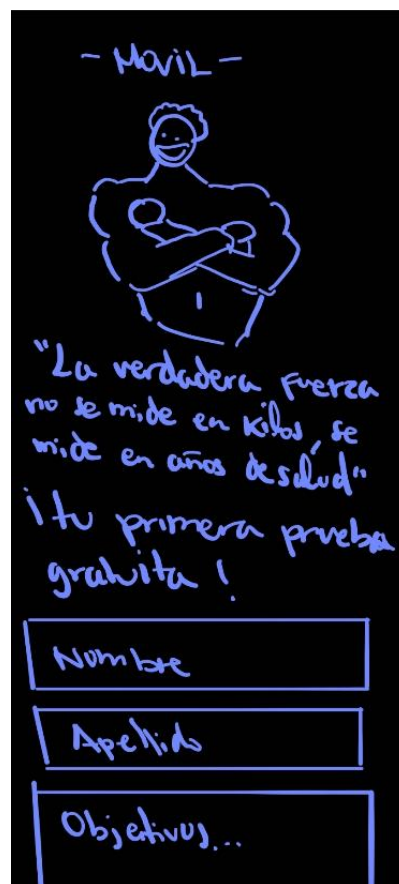
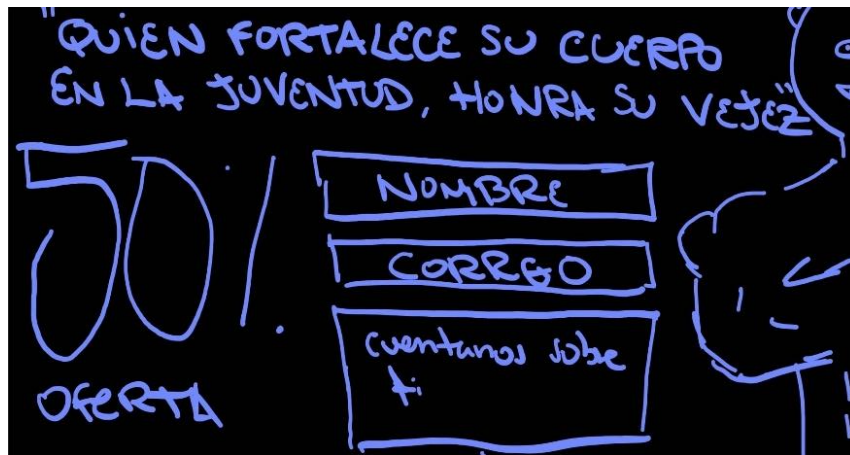


Fig 1. Primer borrador descartado por falta de estructura clara.

Para cumplir con los objetivos del cliente, se rediseñó el boceto aplicando estrictamente las leyes de la Gestalt para asegurar que las cajas de contenido estuviesen claramente definidas y alineadas.

The sketch shows a desktop application layout. At the top left is a box labeled 'LOGO'. To its right is the title 'ORDENADOR'. Below the logo is a large box labeled 'IMG DE FONDO'. Inside this box, on the left, is the text 'Quien fortalece su cuerpo en la juventud, honra su vejez'. On the right side of the 'IMG DE FONDO' box, there is a vertical stack of elements: the text 'Tu Primera Prueba gratis', followed by three input fields labeled 'NOMBRE', 'CORREO', and 'Elige tu disciplina ✓', and finally a large button labeled 'PROBAR GRATIS'.

The sketch shows a mobile application layout. At the top left is a box labeled 'LOGO'. To its right is the title 'Móvil'. Below the logo is a large box labeled 'IMG de fondo'. Inside this box, on the left, is the text '"La verdadera fuerza no se mide en kilos, se mide en años de salud"'. Below this is the text '¡tu primera prueba gratuita!'. On the right side of the 'IMG de fondo' box, there is a vertical stack of elements: three input fields labeled 'NOMBRE', 'CORREO', and 'OBJETIVOS', and finally a large button labeled 'QUIERO MI PRUEBA GRATUITA'.

Fig 2a. Boceto Final Consolidado (Vista Escritorio y Móvil).

4.2. Wireframes y Justificación: Diseño consolidado en Figma .

LOGO

Quien fortalece su
cuerpo en la juventud,
honra su vejez.

ATHLOS FORGE BY SEBAS

Tu Primera Prueba Gratis

NOMBRE

CORREO

QUE QUIERES ENTRENAR

RESERVAR CLASE

LOGO

La verdadera Fuerza no
se mide en kilos...
...se mide en años de
salud.

Tu Primera Prueba Gratis

NOMBRE

CORREO

QUE QUIERES ENTRENAR

RESERVAR CLASE

Fig 2b. Boceto Final Consolidado Figma(Vista Escritorio y Móvil).

4.3. Justificación del Boceto Elegido

Este diseño fue aprobado como base para la implementación por las siguientes razones:

1. **Objetivos del Cliente:** El formulario de captación ("Tu primera prueba gratis") es el elemento protagonista, situado en la zona caliente de la pantalla para maximizar leads.
2. **Coherencia Visual:** El uso de fondo negro con líneas doradas/amarillas establece una estética premium y energética desde el primer vistazo.

También cabe la posibilidad de utilizar la imagen de fondo para que el posible cliente centre su atención directamente en el CTA.

3. Experiencia de Usuario (UX): La estructura es simple. El usuario entiende en menos de 3 segundos qué ofrece la web y qué debe hacer (rellenar el formulario).
4. Impacto Comercial: A diferencia de la competencia analizada, se ha limpiado el menú de navegación para reducir distracciones y enfocar al usuario en la conversión.

4.4. Requisitos y Modelado del Sistema

Para asegurar que el desarrollo se alinee fielmente con las necesidades funcionales de la plataforma y el correcto estándar de la ingeniería del software, se definen y modelan las especificaciones del sistema.

4.4.1. Requisitos Funcionales (RF)

- **RF-01 Registro y Autenticación de Usuarios:** El sistema debe permitir a los usuarios crear cuentas y autenticarse de manera segura a través de un formulario con validación de fortaleza en tiempo real.
- **RF-02 Gestión del Catálogo de Servicios:** El cliente debe poder visualizar las modalidades de entrenamiento disponibles e interactuar dinámicamente con un buscador local sin latencia.
- **RF-03 Gestión de la Cesta de la Compra:** El sistema debe permitir agregar, eliminar y calcular el coste total acumulado de las sesiones en tiempo real utilizando almacenamiento persistente.
- **RF-04 Panel Administrativo (Dashboard):** El usuario con rol administrador debe disponer de una interfaz privada para realizar operaciones CRUD sobre los entrenamientos, usuarios y monitorizar el historial de transacciones comerciales.

4.4.2. Requisitos No Funcionales (RNF)

- **RNF-01 Seguridad en Almacenamiento:** El sistema debe garantizar el hashing criptográfico de contraseñas de usuarios mediante algoritmos unidireccionales seguros (bcryptjs) antes de su persistencia en base de datos.
- **RNF-02 Rendimiento y Latencia:** Las funciones de búsqueda y filtrado de servicios en el cliente deben ejecutarse localmente de forma instantánea (Zero Latency) para catálogos de baja densidad.
- **RNF-03 Portabilidad y Multiplataforma:** El código fuente de la aplicación debe ser compatible para su ejecución nativa en dispositivos

móviles Android (API 24 o superior) e interfaces web de escritorio de forma unificada.

- **RNF-04 Accesibilidad:** Toda la interfaz de usuario interactiva debe cumplir con un ratio de contraste adecuado y ser navegable completamente mediante teclado empleando landmarks semánticos y especificaciones ARIA.

4.5. Landing page

La landing page implementada respeta fielmente el boceto aprobado, mejorando la calidad visual con el uso de un logo profesional y fotografía real.

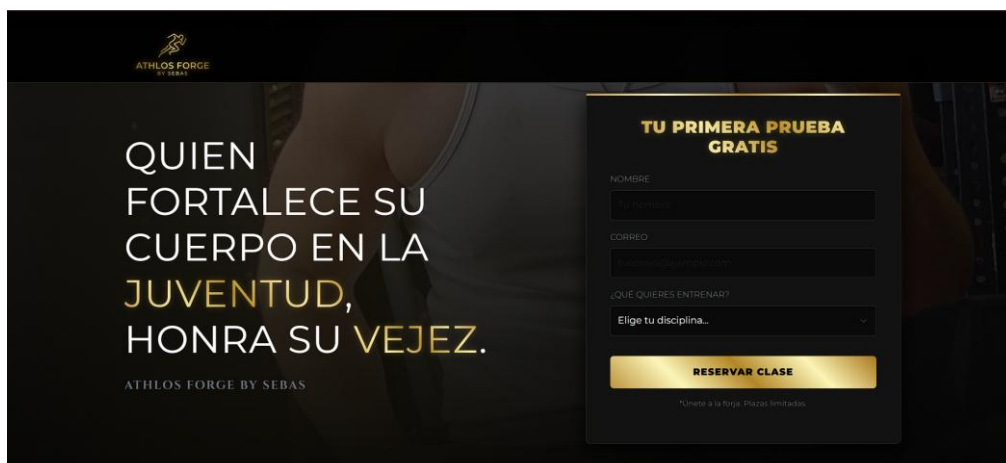
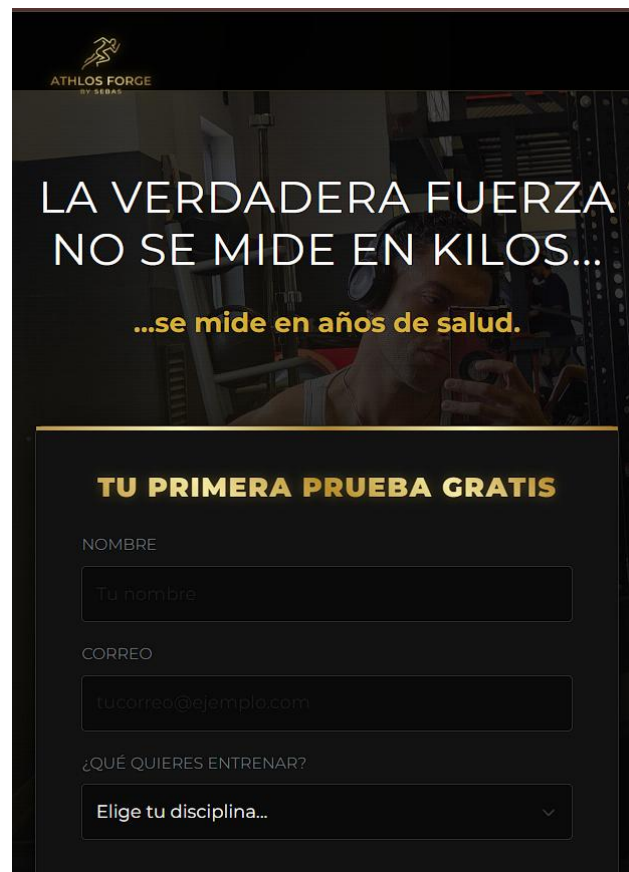
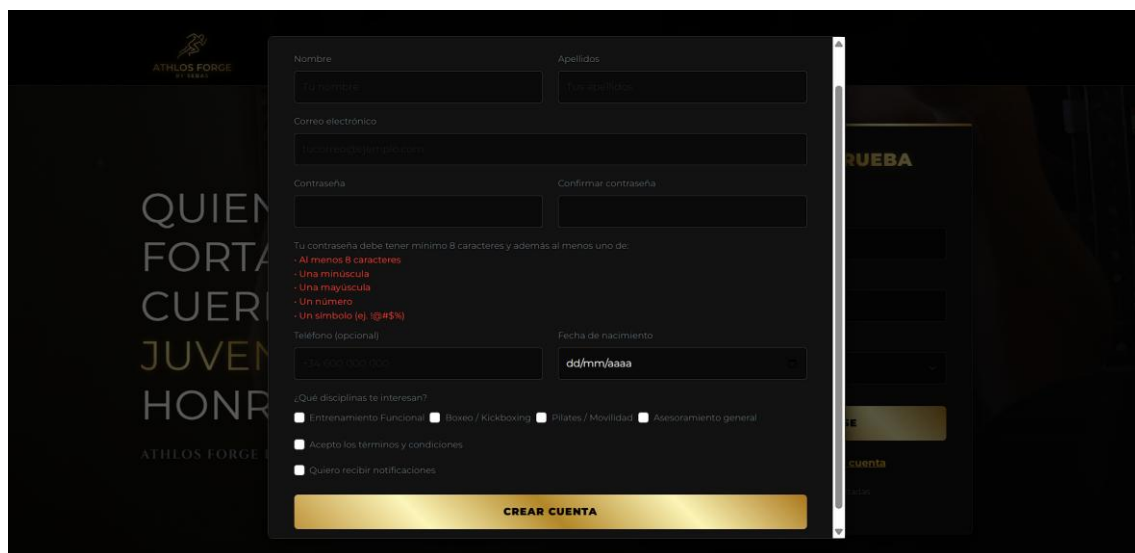


Fig 3. Implementación final de la Landing Page en navegador.



The image shows a mobile landing page for 'ATHLOS FORGE BY SEBAS'. At the top, there's a logo and a background image of a person in a gym. The main text reads 'LA VERDADERA FUERZA NO SE MIDE EN KILOS... ...se mide en años de salud.' Below this, a section titled 'TU PRIMERA PRUEBA GRATIS' contains a form with fields for 'NOMBRE' (with placeholder 'Tu nombre'), 'CORREO' (with placeholder 'tucorreo@ejemplo.com'), and a dropdown for '¿QUÉ QUIERES ENTRENAR?' (with placeholder 'Elige tu disciplina...').

Fig 4. Implementación final de la Landing Page en móvil.



The image shows a mobile registration form for 'ATHLOS FORGE BY SEBAS'. The form is titled 'QUIEN FORTALECE TU CUERPO JUVENIL HONRA TU NOMBRE' and 'ATHLOS FORGE BY SEBAS'. It includes fields for 'Nombre' (with placeholder 'Tu nombre'), 'Apellidos' (with placeholder 'Tu apellido'), 'Correo electrónico' (with placeholder 'Tu correo de correo electrónico'), 'Contraseña' (with placeholder 'Tu contraseña'), and 'Confirmar contraseña' (with placeholder 'Confirma tu contraseña'). Below these fields, there's a list of requirements for the password: 'Tu contraseña debe tener mínimo 8 caracteres y además al menos uno de: - Al menos 8 caracteres, - Una minúscula, - Una mayúscula, - Un número, - Un símbolo (ej. !@#\$%)'. There's also a field for 'Teléfono (opcional)' (with placeholder 'Tu número de teléfono') and a field for 'Fecha de nacimiento' (with placeholder 'dd/mm/aaaa'). Below these fields, there's a section for '¿Qué disciplinas te interesan?' with checkboxes for 'Entrenamiento Funcional', 'Boxeo / Kickboxing', 'Pilates / Movilidad', and 'Asesoramiento general'. There are also checkboxes for 'Acepto los términos y condiciones' and 'Quiero recibir notificaciones'. At the bottom, there's a yellow button labeled 'CREAR CUENTA'.

Fig 5. Implementación final del formulario de registro.

Sin embargo, a la hora de realizar el primer sprint, intente centrarme en el call to action para captar leads, con una prueba gratuita, pero como ya tienen que meter los datos para coger la prueba decidí cambiarlo a un botón radiante de color dorado para que pueda acceder en caso de que no tenga cuenta, y aparte tiene el botón de iniciar sesión en caso contrario, haciéndolo mucho más coherente



Fig 5. Landing page final

4.6 FACTIBILIDAD Y RECURSOS

4.1. Infraestructura Técnica (Software)

Para mantener un margen de rentabilidad alto, se emplearán herramientas de desarrollo de vanguardia con costes de licencia nulos o mínimos:

- Diseño UI/UX: Figma para el prototipado.
- Desarrollo: Visual Studio Code y el framework Bootstrap.
- Recursos Multimedia: Pexels y bancos de imágenes de alta resolución para mantener la estética profesional.

4.2. Inversión Operativa (Hardware y Logística)

El modelo de negocio es altamente eficiente en costes:

- Entrenamientos al aire libre: Minimizan los gastos de infraestructura fija.
- Material: Inversión inicial en equipamiento básico (pesas, bandas, etc.) estimada en menos de 200€.

- Clases Indoor: Alquiler puntual de espacios según demanda, evitando costes fijos de mantenimiento de un local propio.

4.3 Cronograma y desarrollo

El tiempo de entrega de la web es variable y se ajustará mediante un modelo de desarrollo ágil. Este dependerá de:

1. Requerimientos del cliente: Definición final de funcionalidades y secciones.
2. Ciclos de revisión: Reuniones de alineación para asegurar que el producto final cumpla con la visión de "Athlos Forge".

Para poder llevar acabo las historias de usuario se planteara el uso del diagrama de Gannt a continuación, siendo este el esquema que seguiré durante todo el proyecto.



Diagrama de Gannt

1. Investigación y Estrategia	Análisis de Mercado (Competencia)	05/01/2026	10/01/2026	5	100%	Sebas
1. Investigación y Estrategia	Definición de Público Objetivo y Nicho	10/01/2026	13/01/2026	3	100%	Sebas
2. Identidad y Diseño	Diseño de Logotipo (Looka/Dverso)	15/01/2026	22/01/2026	7	100%	Sebas
2. Identidad y Diseño	Definición de Estética Premium Spartan	22/01/2026	25/01/2026	3	100%	Sebas
2. Identidad y Diseño	Wireframes y Prototipado en Figma	26/01/2026	05/02/2026	10	100%	Sebas
3. Desarrollo Frontend	Estructura HTML5 y Grid Bootstrap 5	06/02/2026	13/02/2026	7	100%	Sebas
3. Desarrollo Frontend	Estilos CSS3 (Negro/Dorado)	13/02/2026	18/02/2026	5	100%	Sebas
4. Programación JS/PHP	Clase AutenticacionManager (JS)	18/02/2026	24/02/2026	6	100%	Sebas
4. Programación JS/PHP	Implementación del Carrito de Compra	24/02/2026	04/03/2026	8	100%	Sebas
4. Programación JS/PHP	Sistema de Filtrado Local	04/03/2026	08/03/2026	4	100%	Sebas
4. Programación JS/PHP	Backend PHP y Gestión de Sesiones	08/03/2026	15/03/2026	7	100%	Sebas
5. Gestión Admin	Desarrollo del Admin Dashboard	15/03/2026	22/03/2026	7	100%	Sebas
6. Calidad y QA	Pruebas Automatizadas con Selenium	22/03/2026	27/03/2026	5	100%	Sebas
6. Calidad y QA	Auditoría Accesibilidad WCAG 2.1 (WAVE)	27/03/2026	31/03/2026	4	100%	Sebas
6. Calidad y QA	Inspección Heurística y Ajustes Finales	31/03/2026	04/04/2026	4	100%	Sebas

5. Implementación Técnica (Desarrollo)

5.1. Stack Tecnológico y Arquitectura

Para el desarrollo de la plataforma Athlos Forge, se ha seleccionado un Stack tecnológico robusto que combina el diseño visual con lógica de programación avanzada:

- Frontend (Interfaz y UX):
 - HTML5 & CSS3: Estructura semántica avanzada y estilos personalizados para lograr la estética *Premium* (dorado/negro).
 - Bootstrap 5: Framework fundamental para el diseño *Mobile-First*. Se ha utilizado su sistema de *grid* (12 columnas) y componentes como *Modals* y *Offcanvas* para el carrito.
- Lógica de Cliente (JavaScript ES6+):
 - Programación orientada a objetos (POO) mediante la clase `AutenticacionManager`.
 - Uso de `Async/Await` y la `Fetch API` para la comunicación asíncrona con el servidor, evitando recargas de página y mejorando la fluidez (SPA-like).
 - Manipulación dinámica del DOM para el sistema de filtrado de clases y gestión del carrito de compras.
- Backend y Persistencia (Servidor):
 - PHP: Procesamiento de datos en el lado del servidor, gestión de sesiones seguras y lógica de negocio.
 - Web Storage API (`LocalStorage`): Utilizado para la persistencia del carrito de compras y datos temporales del usuario, garantizando que la información no se pierda al navegar.
- Calidad y Automatización (QA):
 - Selenium : Implementación de pruebas de regresión automatizadas para validar el correcto funcionamiento de los formularios de registro y login bajo distintos escenarios.

5.2. Implementación del carrito de la compra

5.2.1. Descripción General

El módulo de la cesta de compra ha sido desarrollado íntegramente en JavaScript Vanilla (ES6+), prescindiendo de cualquier framework o librería

externa (como jQuery). Se ha diseñado bajo un enfoque de Manipulación Segura del DOM, priorizando el rendimiento y la protección contra ataques de inyección de código.

5.2.2. Arquitectura de Datos

La información de los servicios seleccionados se gestiona mediante un Array de Objetos. Esta estructura permite una manipulación ágil de los datos antes de ser renderizados.

Ejemplo de objeto de producto:

JavaScript

```
{  
  id: 1710864000000, // Identificador único basado en Timestamp  
  nombre: "Plan Olimpo",  
  precio: 49.99  
}
```

5.2.3. Estándares de Manipulación del DOM

Para cumplir con los requisitos técnicos de alta seguridad, la interfaz se genera dinámicamente utilizando exclusivamente métodos nativos de creación de nodos.

A. Creación de Elementos (Sin innerHTML dinámico)

Siguiendo las mejores prácticas, se evita el uso de innerHTML para insertar contenido variable. En su lugar, se emplea un flujo de trabajo de tres pasos:

1. `document.createElement()`: Para generar cada nodo HTML (divs, h6, botones).
2. `textContent`: Para asignar los nombres y precios de forma segura.
3. `appendChild()`: Para ensamblar la estructura jerárquica del carrito.

B. Gestión de Eventos

En lugar de utilizar atributos onclick en el HTML, se ha implementado el método `addEventListener()`. Esto separa completamente la lógica del comportamiento de la estructura del documento, facilitando el mantenimiento del código.

5.2.4. Funcionalidades Principales

Función	Descripción Técnica
agregarAlCarrito()	Captura los datos del plan, genera un ID único y actualiza el array principal.
eliminarDelCarrito()	Utiliza el método <code>.filter()</code> para remover objetos del array por su ID.
renderizarItems()	Itera el array con <code>.forEach()</code> , construye los nodos del DOM y los inyecta en el panel lateral.
calcularTotal()	Emplea <code>.reduce()</code> para obtener la suma exacta de los precios en el array.
actualizarLocalStorage()	Sincroniza el estado del carrito con la memoria del navegador para persistencia de datos.

5.2.5 Persistencia y Estado

Para que el usuario no pierda su selección al recargar la página, el módulo se integra con la Web Storage API:

- Lectura: Al cargar el DOM, el script recupera el JSON almacenado.
- Escritura: Cada modificación en el array dispara una actualización en `localStorage`.

5.3 Sistema de filtrado local

5.3.1. Estrategia de Implementación: Client-Side vs AJAX

Para el buscador de la sección de entrenamientos, se ha optado por un procesamiento local en el cliente en lugar de peticiones asíncronas al servidor (AJAX).

Justificación técnica:

- Rendimiento (Zero Latency): Al tener un catálogo de 8 servicios, realizar una petición al servidor por cada pulsación de tecla generaría un tráfico de red innecesario. El filtrado local es instantáneo.
- Experiencia de Usuario (UX): La búsqueda responde en tiempo real a medida que el usuario escribe, sin tiempos de carga ni *spinners*.

- Eficiencia de Recursos: Se reduce la carga del servidor y el consumo de datos, ya que toda la información necesaria ya reside en el DOM tras la carga inicial de la página.
-

5.3.2. Estructura del Catálogo (Grid System)

El catálogo se ha expandido a 8 modalidades de entrenamiento, organizadas en un sistema de rejilla adaptable de Bootstrap:

Entrenamiento	Precio	Enfoque Principal	Gradiente Visual
GAP	55€	Glúteo, Abdomen, Pierna	Cyan - Azul
Rehabilitación	65€	Fisioterapia	Teal - Verde
Fuerza	70€	Alta Intensidad	Naranja - Rojo
Ciclo/Spinning	50€	Cardio / Resistencia	Rosa - Amarillo
<i>Originales</i>	30€-99€	Boxeo, Funcional, etc.	Dorado Athlos

5.3.3. Lógica del Buscador (Vanilla JS)

El motor de búsqueda utiliza funciones de orden superior de ES6 para manipular la visibilidad de los elementos del DOM de forma eficiente.

Características del algoritmo:

- Case-Insensitivity: Convierte tanto el valor del *input* como el nombre del entrenamiento a minúsculas para evitar errores de coincidencia por mayúsculas.
- Manipulación Dinámica de Clases: En lugar de eliminar elementos, se utilizan clases de utilidad para mostrar/ocultar tarjetas, lo que mantiene la estructura del *layout* estable.
- Contador Dinámico: Se actualiza en cada evento de entrada (input), informando al usuario del número exacto de resultados visibles mediante el método `.querySelectorAll(':not(.d-none)').`

5.3.4. Integración con el Carrito de Compra

A pesar de ser un sistema de búsqueda dinámico, la integración con el módulo `carrito.js` se mantiene intacta:

1. Cada tarjeta filtrada conserva su atributo onclick con la función `agregarAlCarrito()`.
2. Los datos (Nombre y Precio) se pasan como argumentos directamente, asegurando que la persistencia en `localStorage` funcione correctamente sin importar si el elemento está siendo filtrado o no.

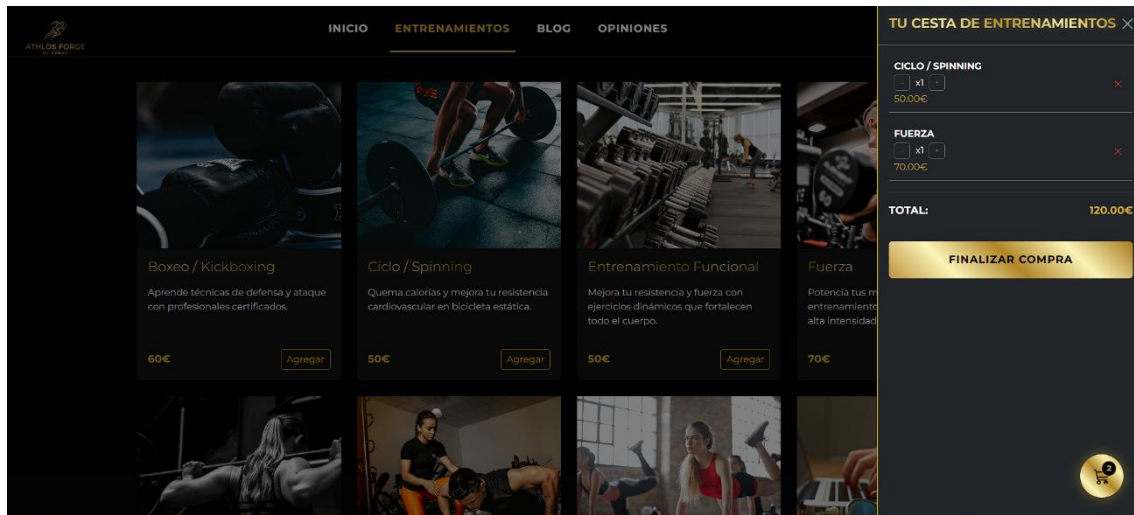
Flujo de Trabajo Técnico (Workflow)

1. Captura del Evento: Escucha de eventos `keyup` o `input` en la barra de búsqueda.
2. Filtrado: Comparación de la cadena introducida con el contenido de los títulos de las tarjetas.
3. Refresco de UI: Actualización del contador y visibilidad de las columnas de Bootstrap (`col-lg-3`).
4. Reset: El botón "Limpiar" restaura el array de elementos a su estado original de visibilidad.

5.3.5. Sistema de Comercio Electrónico y Carrito en Tiempo Real

La plataforma permite la adquisición de servicios de entrenamiento mediante una experiencia de usuario fluida y reactiva.

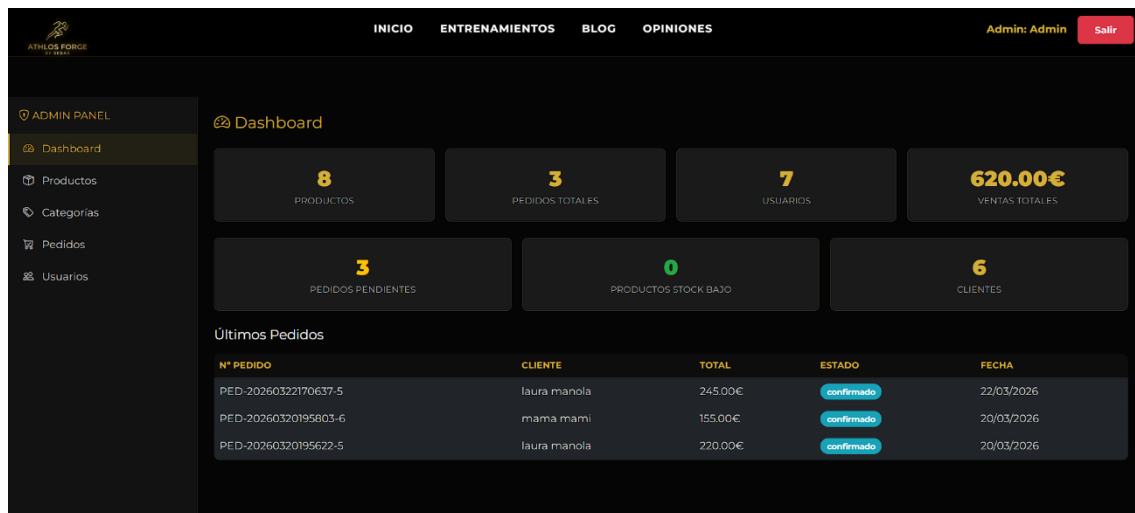
- Gestión Dinámica de Servicios: El catálogo de entrenamientos (Boxeo, Ciclo, Funcional, Fuerza) permite la selección y adición de productos al carrito de forma instantánea mediante JavaScript nativo.
- Actualización en Tiempo Real: Al interactuar con el botón "Agregar", el sistema actualiza automáticamente el contador del icono flotante y el desglose de precios dentro del panel *Offcanvas*.
- Persistencia de Selección: Se utiliza la Web Storage API (`LocalStorage`) para asegurar que los servicios seleccionados por el cliente permanezcan guardados incluso si la página se recarga o se cierra accidentalmente.
- Cálculo Automatizado: El sistema emplea el método `.reduce()` para computar el total de la compra (ej. 165.00€) basándose en los precios de los servicios en el array activo.



5.3.6. Panel de Administración (Admin Dashboard)

Para facilitar la gestión empresarial, se ha desarrollado una interfaz de administración privada que permite el control total sobre los recursos de la web.

- **Dashboard de Métricas:** Vista resumida que muestra indicadores clave de rendimiento (KPIs) en tiempo real, incluyendo:
 - **Productos (8):** Total de entrenamientos disponibles.
 - **Pedidos Totales (3):** Historial de transacciones procesadas.
 - **Usuarios (7):** Control de la base de datos de clientes registrados.
 - **Ventas Totales (620.00€):** Monitorización de ingresos generados.
- **Gestión de Inventario (CRUD):** El administrador tiene la capacidad de modificar o eliminar entrenamientos existentes, así como supervisar productos con stock bajo o pendientes de configuración.
- **Control de Pedidos:** Listado detallado de las últimas transacciones, permitiendo verificar el número de pedido (ej. PED-20260322...), el nombre del cliente, el total y el estado de confirmación.



5.3.7. Gestión de Usuarios y Seguridad

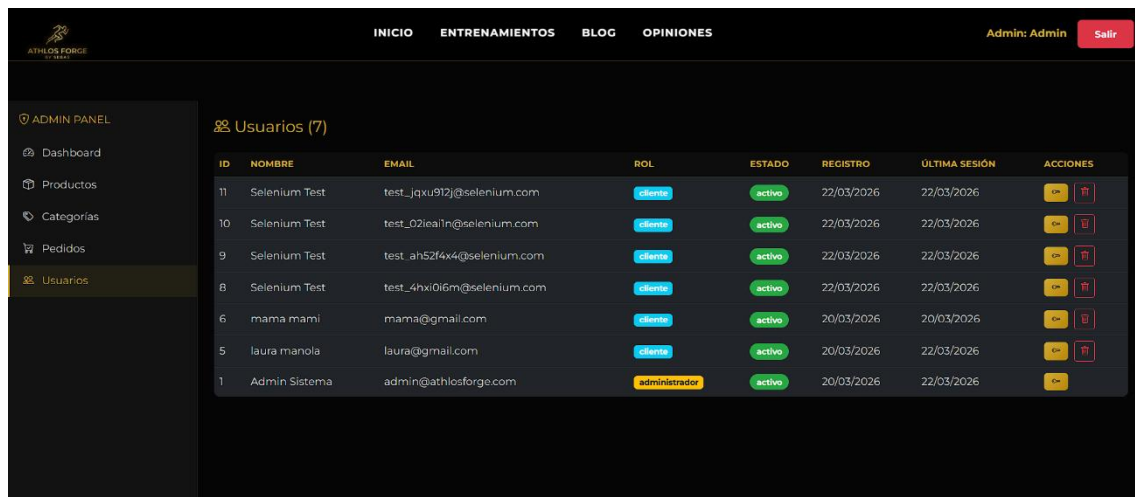
El sistema garantiza un entorno seguro para la administración y la recuperación de acceso.

- **Administración de Cuentas:** Panel específico para visualizar el listado de usuarios con sus respectivos roles (Admin/Cliente), correos electrónicos y fechas de registro.
- **Recuperación de Acceso:** Se ha implementado un flujo de seguridad para la recuperación de contraseña, permitiendo a los usuarios restablecer sus credenciales de acceso de forma autónoma.
- **Trazabilidad de Sesiones:** El sistema registra la "Última Sesión" de cada usuario, facilitando auditorías de seguridad y control de actividad en la plataforma.

5.3.8 Análisis de Ventas y Productos Vendidos

La página integra una vista de reporte para el seguimiento comercial.

- **Historial de Transacciones:** Registro cronológico de los servicios vendidos, asociando cada venta a un cliente específico (ej. "laura manola", "mama mami") para una gestión personalizada.
- **Visualización de Estados:** Uso de etiquetas visuales para identificar pedidos "confirmados", optimizando el flujo de trabajo del administrador al gestionar las nuevas ventas.



The screenshot shows the Admin Panel of the Athlos Forge system. The top navigation bar includes links for INICIO, ENTRENAMIENTOS, BLOG, and OPINIONES, along with a user profile 'Admin: Admin' and a 'Salir' button. The left sidebar contains a menu with options like Dashboard, Productos, Categorías, Pedidos, and Usuarios. The main content area displays a table titled 'Usuarios (7)' with the following data:

ID	NOMBRE	EMAIL	ROL	ESTADO	REGISTRO	ÚLTIMA SESIÓN	ACCIONES
11	Selenium Test	test_jqxu912j@selenium.com	cliente	activo	22/03/2026	22/03/2026	[iconos]
10	Selenium Test	test_02iealn@selenium.com	cliente	activo	22/03/2026	22/03/2026	[iconos]
9	Selenium Test	test_ah52f4x4@selenium.com	cliente	activo	22/03/2026	22/03/2026	[iconos]
8	Selenium Test	test_4hxi0i6m@selenium.com	cliente	activo	22/03/2026	22/03/2026	[iconos]
6	mama mami	mama@gmail.com	cliente	activo	20/03/2026	20/03/2026	[iconos]
5	laura manola	laura@gmail.com	cliente	activo	20/03/2026	22/03/2026	[iconos]
1	Admin Sistema	admin@athlosforge.com	administrador	activo	20/03/2026	22/03/2026	[iconos]

6. Sistemas de validaciones y seguridad

Para garantizar la integridad de los datos y una experiencia de usuario fluida, el sistema implementa validaciones en tiempo real y previo al envío (on-submit) mediante la clase AutenticacionManager.

6.1 Validación de Identidad y Formato

El sistema aplica reglas estrictas para asegurar que la información recogida sea profesional y útil para el marketing:

- **Nombre y Apellidos:** Se limita la entrada a un máximo de dos palabras para cada campo. Esto evita entradas de texto basura y asegura que los datos sean coherentes con un perfil de cliente real.
- **Correo Electrónico:** Se utiliza una expresión regular (Regex) para verificar que la estructura del email sea válida (usuario@dominio.ext), reduciendo errores de escritura.
- **Campos Obligatorios:** Se valida la presencia de género, fecha de nacimiento, dirección y país antes de permitir el envío del formulario.

6.2 Lógica de Seguridad de Contraseñas

Siguiendo los estándares de seguridad modernos, la validación de contraseñas es dinámica y educativa:

- **Criterios de Complejidad:** La contraseña debe cumplir con cinco requisitos mínimos:
 1. Mínimo 8 caracteres.
 2. Al menos una letra minúscula.

3. Al menos una letra mayúscula.
 4. Al menos un número.
 5. Al menos un símbolo especial.
- Feedback Visual (UX): Mediante el método `validarContraseña`, el usuario recibe respuesta inmediata. Los elementos de la interfaz cambian de color (de `text-danger` a `text-success`) y se actualizan con un icono de verificación (`bi-check-circle`) conforme se cumplen las reglas.
 - Indicador de Fortaleza: Se implementa una barra de progreso que clasifica la contraseña como "Débil", "Media" o "Fuerte", incentivando al cliente a elegir credenciales más robustas.

6.3 Validación de Datos Sensibles (Tarjeta de Crédito)

Dada la naturaleza Premium del servicio en Las Rozas, se ha incluido una validación de tarjeta de crédito:

- Validación de Formato: El método `validarTarjeta` utiliza `Regex` para confirmar que el número tenga entre 13 y 19 dígitos, eliminando espacios en blanco automáticamente para mayor comodidad del usuario.
- Lógica Condicional: El campo de tarjeta no es siempre visible; se activa mediante el método `toggleTarjetaField` solo cuando el usuario ha completado los campos de dirección y país, reduciendo la carga cognitiva inicial del formulario.

6.4 Gestión de Errores y Feedback

Para evitar la frustración del usuario, el sistema incluye un manejador centralizado de errores:

- Método `mostrarError`: Si alguna validación falla, se despliega un mensaje claro en un elemento de alerta (`alertElement`), eliminando la clase `none` de Bootstrap y realizando un `scroll` suave (`smooth scroll`) hacia la advertencia para que el usuario sepa exactamente qué corregir.

7. TESTS FUNCIONALES AUTOMATIZADOS (SELENIUM)

Para garantizar la robustez del código cliente y la correcta implementación de las validaciones, se ha desarrollado una suite de pruebas automatizadas utilizando Selenium. Estas pruebas simulan la interacción de usuarios reales

con el formulario de registro, asegurando que el sistema responda correctamente ante flujos de datos válidos antes de su despliegue.

7.1. Descripción del Entorno de Pruebas

El sistema de pruebas está diseñado para ejecutarse en un entorno local y verifica el ciclo completo de registro.

- Herramienta: Selenium WebDriver.
- Lenguaje del Script: Python (3.8+).
- Navegador Objetivo: Google Chrome (gestionado automáticamente vía ChromeDriver).
- Servidor de Pruebas: VS Code Live Server (Puerto 5500).

7.2. Flujo de Ejecución Automatizado

El script de prueba (run_tests_multiples.ps1) realiza la siguiente secuencia de acciones de forma autónoma:

1. Inicialización del Entorno: Detecta la instalación de Python, crea un entorno virtual (.venv) e instala las dependencias necesarias (Selenium, WebDriver Manager) si no existen.
2. Lanzamiento del Navegador: Abre instancias de Google Chrome de manera secuencial.
3. Simulación de Usuario: Navega a <http://127.0.0.1:5500/> e interactúa con el DOM para rellenar los campos del formulario (Nombre, Email, Password, Confirmación).
4. Validación de Resultados: Envía el formulario y verifica mediante selectores CSS que aparezca el mensaje de éxito o la alerta correspondiente.

7.3. Escenario de Prueba: Registro Múltiple

Actualmente, el script ejecuta un escenario de carga con 3 usuarios de prueba distintos para verificar la estabilidad del formulario ante registros consecutivos:

1. Usuario 1: Datos estándar válidos.
2. Usuario 2: Datos válidos con variantes en el formato de nombre.
3. Usuario 3: Datos válidos con correos de dominios diferentes.

7.4. Guía de Ejecución Técnica y Despliegue de Tests

Para garantizar la replicabilidad de las pruebas en cualquier entorno de desarrollo (portabilidad), se ha estandarizado un flujo de trabajo automatizado que prepara el entorno y ejecuta la suite de Selenium.

7.4.1. Requisitos Previos del Sistema

Antes de iniciar la batería de pruebas, la máquina de destino debe cumplir con los siguientes componentes:

- **Intérprete de Lenguaje:** Tener instalado Python 3.8 o superior, asegurando que la opción *"Add Python to PATH"* esté activada durante la instalación.
- **Servidor de Desarrollo:** El proyecto debe estar servido localmente (recomendado *VS Code Live Server*) bajo la dirección `http://127.0.0.1:5500`, que es el punto de entrada configurado en los scripts de prueba.
- **Navegador:** Disponer de Google Chrome, ya que el script utiliza *WebDriver Manager* para gestionar automáticamente el binario de *ChromeDriver*.

7.4.2. Automatización del Entorno (PowerShell)

Se ha desarrollado un script controlador en PowerShell (`run_tests_multiples.ps1`) que simplifica la corrección y ejecución, realizando de forma autónoma la creación de entornos virtuales (`.venv`) y la instalación de dependencias necesarias.

Pasos para la ejecución:

1. **Localización del Proyecto:** Navegar mediante la terminal al directorio raíz del ecosistema:

PowerShell

```
cd "htdocs/Athlos Forge by Sebas"
```

2. **Configuración de Permisos:** Habilitar la ejecución de scripts en la sesión actual para permitir el despliegue del entorno virtual:

PowerShell

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

3. **Lanzamiento de la Batería de Tests:** Ejecutar el controlador para iniciar la simulación de los 3 escenarios de usuario (estándar, variante de formato y dominios externos):

PowerShell

.\\tests\\selenium\\run_tests_multiples.ps1

7.4.3. Interpretación de Resultados

Al finalizar la secuencia, la consola generará un Reporte de Salida Técnico con el estado de cada aserción:

- Estado PASS: Indica que el DOM respondió con los selectores de éxito esperados tras las validaciones de AutenticacionManager.
- Estado FAIL: Se detalla el campo o la validación (Regex, longitud o coincidencia) que ha fallado en la integración.

8. AUDITORÍA Y VERIFICACIÓN DE ACCESIBILIDAD (WCAG 2.1)

8.1. Metodología de Evaluación

Para garantizar que Athlos Forge sea una plataforma inclusiva, se ha realizado una auditoría de accesibilidad siguiendo las pautas WCAG 2.1 Nivel A.

La herramienta principal utilizada para este diagnóstico ha sido WAVE (Web Accessibility Evaluation Tool). Esta herramienta nos ha permitido identificar visualmente errores de contraste, falta de etiquetas alt, y problemas en la estructura jerárquica de los encabezados directamente sobre la interfaz de la landing page.

8.2. Registro de Errores Detectados y Subsanaos

Tras el análisis con WAVE, se detectaron y corrigieron 12 puntos críticos. A continuación se detallan los más relevantes:

A. Contenido No Textual (Criterio 1.1.1)

- Problema: El logo principal y las imágenes de los planes de entrenamiento carecían de texto alternativo descriptivo.
- Solución: Se implementó el atributo alt="Athlos Forge - Forja de Campeones" para asegurar que los lectores de pantalla identifiquen la marca.

B. Navegación e Información (Criterio 1.3.1)

- Problema: Los menús y secciones no tenían roles definidos, dificultando la navegación estructural.
- Solución: Se añadieron etiquetas ARIA (role="navigation", aria-label="Navegación principal") y se definieron los *landmarks* de HTML5.

C. Operabilidad mediante Teclado (Criterio 2.1.1)

- Problema: El botón flotante de la cesta de compra era un <div> y no era "alcanzable" mediante la tecla Tabulador.
- Solución: Se transformó el elemento en un <button> con aria-controls, permitiendo que usuarios con movilidad reducida puedan abrir su cesta sin usar el ratón.

8.3. Mejoras en la Interfaz Dinámica (ARIA)

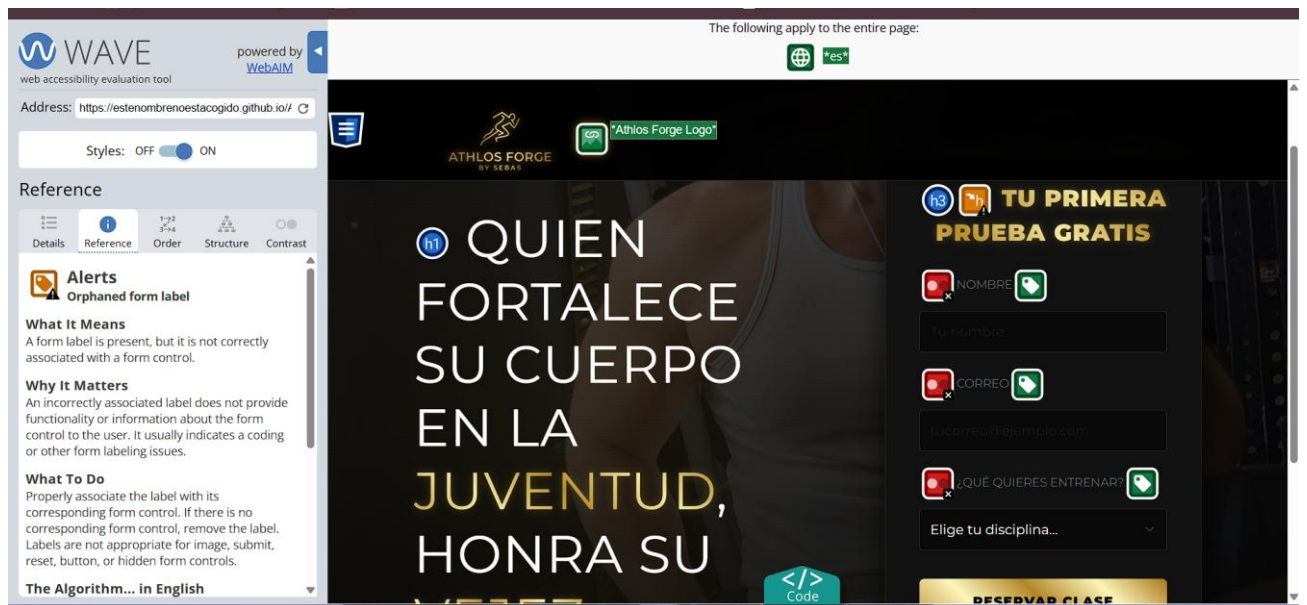
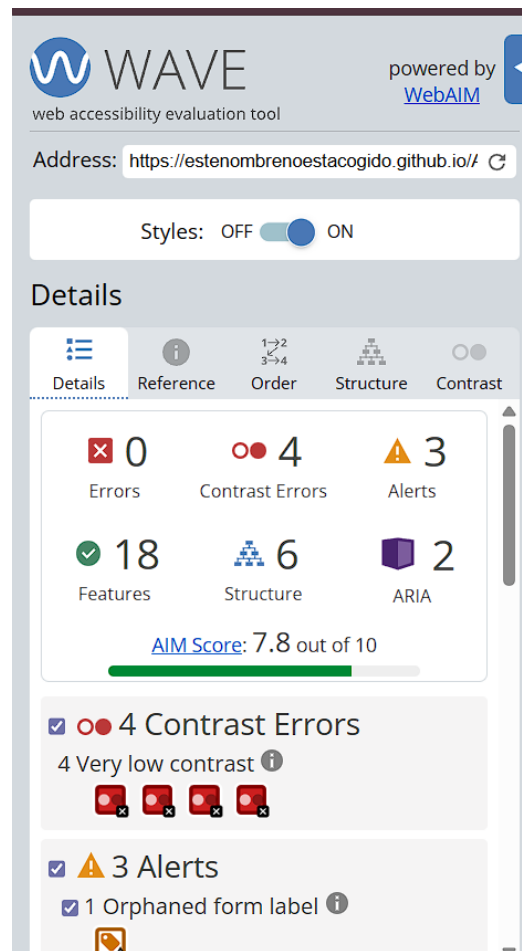
Dado que la web utiliza cambios de estado en tiempo real (como el contador del carrito y los mensajes de bienvenida), se han aplicado Regiones Live:

Elemento	Atributo ARIA aplicado	Propósito
Contador Carrito	aria-live="polite"	Anuncia al usuario cuando se añade un nuevo plan sin interrumpir su lectura.
Total Compra	role="status"	Informa automáticamente del nuevo precio total tras una modificación.
Modal Registro	role="dialog"	Indica al lector de pantalla que se ha abierto una ventana emergente que requiere atención.

8.4. Matriz de Conformidad Final

Tras las correcciones, el estado de cumplimiento verificado con WAVE es el siguiente:

- Perceptibilidad: CUMPLIDO (Textos alternativos y estructura clara).
- Operabilidad: CUMPLIDO (Navegación por teclado optimizada).
- Comprensibilidad: CUMPLIDO (Mensajes de error y etiquetas claras).
- Robustez: CUMPLIDO (Uso de estándares HTML5 y ARIA).



9. INFORME DE VERIFICACIÓN DE LA USABILIDAD: ATHLOS FORGE

Métodos elegidos: Inspección Heurística y Test A/B.

9.1. Método: Inspección Heurística

Se han aplicado 5 de las heurísticas de Jakob Nielsen sobre la interfaz de Athlos Forge.

- Heurística: Estética y diseño minimalista
 - Análisis: La web sigue una línea visual de alto contraste (dorado sobre negro), eliminando cualquier elemento decorativo que no aporte valor.
 - Resultado: Cumple. El diseño refuerza la identidad de marca "Spartan" sin saturar al usuario.
- Heurística: Visibilidad del estado del sistema
 - Problema detectado: Al pulsar "Agregar", el cambio en el contador del carrito es sutil y puede pasar desapercibido si el usuario no mira el botón flotante.
 - Propuesta de mejora: Implementar algún tipo de animación al carrito para que el cliente visualice el nuevo entrenamiento agregado, además de mejorar la notificación .
- Heurística: Consistencia y estandarización
 - Análisis: Se utilizan iconos estándar (carrito de compra) y botones con comportamientos predecibles en toda la landing page.
 - Resultado: Cumple. El usuario reconoce los elementos de interacción por su parecido con otros sitios de e-commerce.
- Heurística: Prevención de errores
 - Análisis: El formulario de registro incluye validaciones en tiempo real (longitud de contraseña, formato de email y tarjeta) que bloquean el envío de datos erróneos.
 - Resultado: Cumple. Se minimiza la posibilidad de que el usuario envíe información inválida al servidor.
- Heurística: Control y libertad del usuario
 - Análisis: Dentro del lateral del carrito (Offcanvas), el usuario tiene la opción de eliminar productos individuales mediante el botón "X".
 - Resultado: Cumple. El usuario puede deshacer acciones de compra de forma rápida y sencilla.

9.2. Método: Test A/B

Se plantea una mejora sobre el elemento principal de conversión de la página.

- Elemento: Botón de llamada a la acción (CTA) en las tarjetas de entrenamientos.
- Versión A (Diseño Actual): Botón con el texto "Agregar".
- Versión B (Propuesta): Botón con el texto "¡EMPEZAR MI TRANSFORMACIÓN!".
- Resultado esperado: Se espera que la Versión B mejore el número de clics (CTR) y la claridad.
- Justificación: El uso de un lenguaje orientado a resultados y un verbo de acción genera un mayor compromiso emocional en el usuario que un texto puramente descriptivo como "agregar".

10. Guía de Despliegue, manual de instalación y entorno de pruebas

Este apartado describe de manera pormenorizada la arquitectura de directorios del proyecto **Athlos Forge by Sebas / SEAL**, los requisitos del sistema para su correcta ejecución, el procedimiento paso a paso para el despliegue de las dos ramas tecnológicas desarrolladas (arquitectura monolítica tradicional y migración SPA con React) y la metodología para la ejecución del bloque de pruebas automatizadas.

10.1. Estructura de Directorios del Proyecto

Para garantizar la mantenibilidad y modularidad del software, se ha seguido una organización estricta separando las capas de presentación, lógica de negocio, persistencia y el entorno de aseguramiento de la calidad (QA):

Athlos Forge by Sebas/

—	index.html # Landing page principal (UI de captación)
—	entrenamientos.html # Catálogo interactivo de servicios y actividades
—	login.html # Módulo de autenticación y registro de usuarios
—	blog.html # Sección de artículos de salud y longevidad
—	opiniones.html # Módulo de reseñas y feedback de clientes

```

|— mis-pedidos.html      # Historial transaccional y pedidos del cliente
|— admin.html           # Panel de control de administración (Dashboard
CRUD)
|
|— css/
|   └— style.css        # Hojas de estilo globales (Tema premium, paleta
#D4AF37)
|
|— js/
|   └— autenticacion.js  # Gestión de sesiones del lado del cliente y Web
Storage
|   └— carrito.js       # Lógica transaccional síncrona y asíncrona del
carrito
|   └— admin.js         # Controladores del panel de administración y
peticiones fetch
|
|— php/
|   └— api.php          # API REST de la aplicación (45+ endpoints
integrados)
|   └— db.php           # Capa de abstracción de datos (PDO), cifrado y
sesiones
|
|— db/
|   └— schema.sql       # Definición de la base de datos (12 tablas,
procedimientos y vistas)
|
|— img/                 # Almacenamiento de recursos gráficos y multimedia
optimizados
|
|— tests/               # Suite de Aseguramiento de la Calidad (QA
Automatizado)
|   └— confest.py       # Configuración global y fixtures de inicialización de
Pytest

```

```

|   └─ test_02_login.py      # Batería de 9 tests para el flujo de autenticación
|   └─ test_05_carrito.py    # Batería de 10 tests para la persistencia y flujo
del carrito
|   └─ requirements.txt      # Manifiesto de dependencias del entorno de
testing en Python
|   └─ reporte.html          # Reporte visual detallado generado tras el testing
|
└─ Documentacion Proyecto Final.pdf

```

10.2. Requisitos Previos del Sistema

Para el correcto despliegue local de la plataforma, el entorno de desarrollo debe contar con los siguientes componentes de software instalados y configurados:

- **Servidor Local:** Stack XAMPP habilitado con los módulos del servidor HTTP Apache y el sistema de gestión de bases de datos relacionales MySQL activos.
- **Entornos de Ejecución:** Python versión 3.10 o superior (para la automatización QA) y Node.js versión 18 o superior (requerido únicamente para el entorno de compilación de React).
- **Navegador Web:** Google Chrome estable para la interacción visual del usuario y la simulación controlada del WebDriver de Selenium.
- **Control de Versiones:** Cliente Git para la conmutación de ramas de desarrollo (*branches*).

10.3. Procedimiento de Despliegue de la Aplicación

10.3.1. Versión Monolítica Tradicional (HTML5 / JavaScript ES6+ / PHP)

Esta arquitectura representa el núcleo inicial del proyecto, apoyándose fuertemente en PHP para la lógica del servidor, persistencia de sesiones seguras y renderizado ágil mediante componentes HTML5 nativos combinados con estilos personalizados basados en Bootstrap 5.3.

1. **Conmutación de Entorno:** Asegurar la permanencia en la rama principal mediante la ejecución en la terminal de comandos:

Bash

```
git checkout main
```

2. **Inicialización del Servidor Local:** Levantar los servicios de Apache y MySQL desde el panel de control de XAMPP.

3. **Aprovisionamiento de la Base de Datos:**

- Acceder al gestor web a través de la URL:
http://localhost/phpmyadmin
- Declarar un nuevo esquema de base de datos bajo el identificador único athlos_forge.
- Utilizar el módulo "Importar" seleccionando el script estructurado ubicado en db/schema.sql y ejecutar para crear las 12 tablas con sus respectivos procedimientos almacenados y datos maestros precargados.

4. **Acceso a la Aplicación:** Abrir el navegador e invocar la raíz del proyecto en la dirección local: http://localhost/Athlos Forge by Sebas/

10.3.2. Versión de Arquitectura Desacoplada (Vite + React 19)

Esta variante tecnológica corresponde a la evolución estratégica y migración del proyecto hacia una Single Page Application (SPA), optimizando la reactividad de la interfaz y la experiencia del usuario (UX) mediante un consumo eficiente de la API REST nativa en PHP.

1. **Conmutación de Entorno:** Cambiar el espacio de trabajo hacia la rama específica de la migración:

Bash

```
git checkout sprint-3-react-migration
```

2. **Preparación del Servidor Backend:** Al tratarse de una arquitectura desacoplada, el servidor Apache y MySQL de XAMPP deben continuar en ejecución para atender las peticiones remotas distribuidas por la capa de datos.
3. **Instalación de Dependencias Node:** Acceder al directorio raíz mediante consola e instalar los módulos requeridos por la aplicación:

Bash

```
cd "c:\xampp\htdocs\Athlos Forge by Sebas"
```

```
npm install
```

4. **Lanzamiento del Servidor de Desarrollo:** Ejecutar el empaquetador de módulos Vite en modo de desarrollo ágil:

Bash

```
npm run dev
```


El sistema desplegará de forma automática el servidor local mapeado en el puerto por defecto: `http://localhost:3000`.

10.4. Protocolo de Pruebas Automatizadas y QA (Selenium + Pytest)

Con el objetivo de garantizar la robustez del código ante cambios drásticos (regresiones), se ha desarrollado una suite exhaustiva de pruebas automáticas que simulan el comportamiento real del usuario sobre flujos críticos del negocio.

1. **Navegación al Entorno de Pruebas:** Posicionar la terminal dentro del directorio que aloja los scripts de verificación:

Bash

```
cd "c:\xampp\htdocs\Athlos Forge by Sebas\tests"
```

2. **Instalación del Entorno de Aislamiento:** Instalar las librerías necesarias especificadas en el archivo de requerimientos (Selenium, Pytest y módulos de reporte):

Bash

```
pip install -r requirements.txt
```

3. **Ejecución Completa de la Suite:** Lanzar la suite de pruebas configurando la generación en paralelo de un reporte enriquecido e independiente en formato HTML5:

Bash

```
python -m pytest test_02_login.py test_05_carrito.py -v --html=reporte.html --self-contained-html
```

4. **Métricas de Calidad Esperadas:** El resultado satisfactorio del análisis de integración continua debe arrojar una tasa de éxito limpia, reflejada textualmente bajo la métrica: 16 passed, 2 skipped. El archivo de auditoría visual interactivo resultante quedará disponible de manera permanente para su consulta técnica en `tests/reporte.html`.

11. Proyecto alternativo Athlos App

La aplicación AthlosApp es una plataforma de entrenamiento personal desarrollada con un stack moderno orientado a web y mobile.

Tecnologías utilizadas

- **Frontend:** [React 19](#) con Vite para desarrollo rápido y build optimizado.
- **Mobile / híbrido:** [Capacitor](#) para empaquetar la app como aplicación Android y usar plugins nativos ([@capacitor/app](#), [@capacitor/camera](#), [@capacitor/toast](#)).
- **UI y componentes:** lucide-react para iconografía, componentes propios para splash screen, cabecera de marca, picker de paletas, toast y gestión de motivación.
- **Gráficas y reportes:** recharts para seguimiento visual de progreso y [html2pdf.js](#) para exportar planes de entrenamiento a PDF.
- **Backend/DB:** Firebase Firestore con persistencia offline y sincronización, usando la configuración de [initializeFirestore](#) con [persistentLocalCache](#).
- **Seguridad:**
 - bcryptjs para hashing de contraseñas.
 - ajv para validación de esquemas JSON.
 - Módulos propios de seguridad: passwordManager.js, tokenManager.js, sanitization.js, validationSchemas.js.
 - Validación de variables de entorno en [validateEnv.cjs](#).
- **Build y scripts:**
 - npm run dev para desarrollo.
 - npm run build para producción.
 - npx cap sync, npx cap open android y npm run cap:build para sincronizar y compilar Android.

Funcionamiento de la app

AthlosApp arranca con un **splash screen** y conserva el estilo premium de la marca. El usuario puede:

- elegir entre varias **paletas de colores** personalizadas,
- acceder a un **dashboard de entrenamiento**,
- gestionar **clientes y planes de entrenamiento**,
- controlar **series, repeticiones y volumen**,

- ver **estadísticas de progreso**,
- comparar fotos de evolución,
- generar y descargar **informes PDF**.

La app también incluye:

- gestión de sesión con **tokens JWT** y almacenado seguro,
- protección contra XSS y entradas inválidas,
- un **botón de retroceso seguro** con doble pulsación y notificación,
- soporte offline mediante caché local y Firestore,
- administrador de frases motivacionales actualizado en tiempo real.

Flujo principal

1. Inicializa **Firebase** y carga la configuración del proyecto.
2. Valida y protege los datos con los módulos de seguridad.
3. Muestra el **splash screen** y luego el contenido principal.
4. Permite el acceso a componentes clave:
 - cabecera de marca,
 - selector de tema,
 - gestor de rutinas,
 - visor de progreso,
 - comparador de fotos,
 - panel admin.
5. Guarda y sincroniza datos con **Firestore**, manteniendo la app funcional incluso sin conexión.
6. Exporta planes de entrenamiento a **PDF** y ofrece retroalimentación mediante notificaciones toast.

12. CONCLUSIÓN FINAL

La culminación de Athlos Forge by Sebas no es solo el fin de un desarrollo de software, sino la materialización de un proceso de aprendizaje intensivo y autogestión. A continuación, se detallan las reflexiones finales sobre este camino:

12.1. El Desafío de la Autogestión Unipersonal

Desarrollar un ecosistema completo que abarca desde el *branding* y la psicología del color hasta la arquitectura de bases de datos y el testing automatizado ha sido un reto considerable. Al no contar con departamentos especializados, he tenido que asumir los roles de Diseñador UI/UX, Desarrollador Full-Stack y Analista de Calidad (QA) simultáneamente esta perspectiva integral de todas las áreas del proyecto me ha permitido entender la importancia de la cohesión entre cada línea de código y cada elemento visual.

12.2. Resiliencia ante el Error

El mundo de la informática se define por la resolución de problemas. Durante el desarrollo, surgieron numerosos obstáculos técnicos: errores de validación, conflictos en las sesiones de PHP y fallos en los selectores de Selenium. Sin embargo, la verdadera satisfacción no reside en la ausencia de fallos, sino en la capacidad de persistir y depurar. Cada error fue una lección que obligó a una autoexigencia constante, transformando la frustración inicial en una paciencia metódica necesaria para alcanzar el estándar de calidad actual.

12.3. Potencial Laboral y Académico

Académicamente, el proyecto demuestra el dominio de tecnologías modernas como JavaScript ES6+, Bootstrap 5 y la automatización con Python.

Laboralmente, se entrega una herramienta real:

- **Gestión en Tiempo Real:** Un carrito de compra funcional que garantiza persistencia mediante *Web Storage*.
- **Control Administrativo:** Un *Dashboard* profesional que permite la monitorización de ventas y usuarios, preparado para la explotación comercial.
- **Escalabilidad:** Una estructura sólida pensada para el lanzamiento oficial de la aplicación en 2026.

12.4. Líneas de Trabajo Futuras y Propuestas de Mejora

Tras consolidar el desarrollo iterativo de las fases actuales del proyecto, se plantean las siguientes líneas de evolución tecnológica para expandir el ciclo de vida de la plataforma Athlos:

1. **Unificación Completa de la Arquitectura:** Migrar los módulos nativos e independientes desarrollados en Vanilla JS de la landing page inicial hacia la arquitectura modular SPA estructurada en React 19 con Vite, centralizando el estado de la aplicación mediante Context API.
2. **Integración de Pasarela de Pagos Real:** Sustituir la lógica de validación condicional y simulación local del formulario de tarjetas de crédito por una integración segura del lado del servidor mediante la API de Stripe o PayPal Checkout, implementando webhooks para la confirmación síncrona de pedidos.
3. **Optimización del Despliegue en Producción Móvil:** Completar la firma digital de la APK compilada nativamente a través de Android Studio y configurar los flujos de distribución en entornos cerrados de prueba dentro de Google Play Console mediante Capacitor.
4. **Módulo Avanzado de Inteligencia Artificial:** Desarrollar un agente de recomendación en el backend que analice las métricas de progreso de la colección de Firestore para generar sugerencias automatizadas de macros alimenticias y variaciones de volumen en las rutinas de los clientes en tiempo real.

12.4. Reflexión Final

En definitiva, Athlos Forge es mi pequeño bebe, me ha hecho replantearme la vida infinidad de veces y me ha hecho pensar si ha sido buena idea realizar el TFG yo solo.

La falta de departamentos ha hecho que tenga que aprender de todo un poco para realizar el proyecto de forma coherente, y dado que sigo en el sprint 2

seguiré planteándomelo hasta que llegue el sprint final, de momento no me arrepiento de haber empezado, la maravillosa aventura de buscar errores constantes y dolores de cabeza sin ningún hombro en el que llorar, ya que cuando apruebe todo el módulo, lloraré de felicidad.

Gracias mundo de la informática por ponerme a prueba.

13. BIBLIOGRAFÍA Y WEBGRAFÍA

13.1. Frameworks y Lenguajes de Programación

- Bootstrap 5.3: Documentación oficial para el diseño de interfaces *responsive* y componentes UI. Recuperado de: <https://getbootstrap.com/>.
- JavaScript MDN Web Docs: Consultas sobre manipulación del DOM, funciones de orden superior (filter, reduce) y Web Storage API. Recuperado de: <https://developer.mozilla.org/>.
- PHP Manual: Referencia técnica para la gestión de sesiones y procesamiento de formularios en el servidor. Recuperado de: <https://www.php.net/manual/es/>.

13.2. Diseño y Prototipado (UI/UX)

- Figma: Herramienta utilizada para la creación de *wireframes* y el diseño de la interfaz final de alta fidelidad. <https://www.figma.com/>.
- Looka: Plataforma de inteligencia artificial empleada para la exploración inicial y generación de conceptos del logotipo. <https://looka.com/>.
- Dverso Laundry: Herramienta especializada para la optimización y eliminación de fondos en recursos gráficos. <https://dverso.com/laundry/>.

13.3. Calidad, Accesibilidad y Testing

- Selenium WebDriver: Documentación para la automatización de navegadores y creación de scripts de prueba en Python. <https://www.selenium.dev/>.
- WAVE (Web Accessibility Evaluation Tool): Suite de herramientas para la auditoría de cumplimiento de las pautas WCAG 2.1. <https://wave.webaim.org/>.
- Leyes de la Gestalt: Aplicación de principios de proximidad y cierre para la jerarquía visual de la *landing page*...
- Heurísticas de Nielsen: Referencia para la inspección de usabilidad y mejora de la interfaz de usuario.

13.4. Recursos Multimedia y Activos

- Pexels / Unsplash: Bancos de imágenes de alta resolución utilizados para las fotografías de entrenamiento y el blog. <https://www.pexels.com/>.
- Google Fonts: Implementación de las familias tipográficas *Montserrat* e *Inter* para el refuerzo de la identidad visual. <https://fonts.google.com/>.

14. Glosario Tecnico

- **DOM (Document Object Model):** Interfaz de programación para documentos HTML que representa la estructura de la página y permite la manipulación dinámica de sus elementos y nodos mediante código.
- **SPA (Single Page Application):** Aplicación web que carga una sola página HTML e interactúa con el usuario de manera dinámica actualizando el DOM asincrónicamente sin recargar el navegador.
- **Capacitor:** Framework de código abierto desarrollado por Ionic que permite empaquetar aplicaciones web modernas (HTML, CSS, JS/React) dentro de contenedores nativos para su despliegue en iOS y Android.
- **Vite:** Herramienta de empaquetado y entorno de desarrollo frontend de última generación que optimiza tiempos de compilación mediante módulos ESM nativos del navegador.
- **Vulnerabilidad XSS (Cross-Site Scripting):** Tipo de ataque de inyección en el que se introducen scripts maliciosos en sitios web benignos y de confianza aprovechando la manipulación insegura del DOM.
- **ARIA (Accessible Rich Internet Applications):** Especificación técnica publicada por el W3C que define formas de hacer que el contenido web dinámico y los componentes interactivos sean más accesibles para personas con discapacidades.
- **CRUD:** Acrónimo que define las cuatro funciones fundamentales para la persistencia y gestión de entidades en un sistema de información: Crear, Leer, Actualizar y Borrar (*Create, Read, Update, Delete*).
- **Firestore:** Base de datos relacional orientada a documentos de tipo NoSQL proporcionada de forma serverless por Google Firebase, optimizada para la sincronización de datos en tiempo real.